HASTE: Training-Free Video Diffusion Acceleration via Head-Wise Adaptive Sparse Attention

Xuzhe Zheng[1*], **Yuexiao Ma**[1*], **Jing Xu**[1], **Xiawu Zheng**[1], **Rongrong Ji**[1], **Fei Chao**[1,✉]

[1]Key Laboratory of Multimedia Trusted Perception and Efficient Computing, Ministry of Education of China, Xiamen University, 361005, P.R. China

[*] Equal contribution
✉ **Corresponding Author**

ABSTRACT

Diffusion-based video generation has advanced substantially in visual fidelity and temporal coherence, but practical deployment remains limited by the quadratic complexity of full attention. Training-free sparse attention is attractive because it accelerates pretrained models without retraining, yet existing online top-p sparse attention still spends non-negligible cost on mask prediction and applies shared thresholds despite strong head-level heterogeneity. We show that these two overlooked factors limit the practical speed-quality trade-off of training-free sparse attention in Video DiTs. To address them, we introduce a head-wise adaptive framework with two plug-in components: Temporal Mask Reuse, which skips unnecessary mask prediction based on query-key drift, and Error-guided Budgeted Calibration, which assigns per-head top-p thresholds by minimizing measured model-output error under a global sparsity budget. On Wan2.1-1.3B and Wan2.1-14B, our method consistently improves XAttention and SVG2, achieving up to 1.93× speedup at 720P while maintaining competitive video quality and similarity metrics.

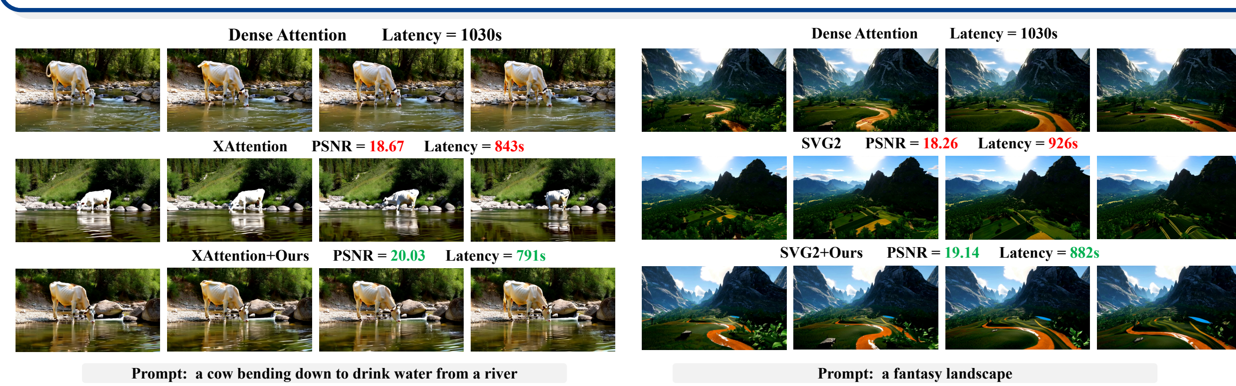


Figure 1. Qualitative comparison on Wan2.1-14B text-to-video generation at 480P. We compare dense attention with two representative sparse-attention baselines, XAttention [35] and SVG2 [36], and their variants enhanced by our method. These results show that our method better preserves the video quality while further improving inference efficiency.

1 Introduction

Video diffusion transformers [15, 23] have become a foundation of modern video generation, driving major advances in visual fidelity, temporal coherence, and long-horizon synthesis [11, 21, 28, 37]. As these systems move toward longer videos and higher resolutions, improving the efficiency of Video DiTs becomes increasingly important for practical deployment [31, 32].

The main challenge is that Video DiTs are expensive along two dimensions. On the one hand, diffusion generation requires multi-step denoising, so the network must be executed repeatedly throughout the sampling trajectory [9, 26]. On the other hand, video inputs contain long token sequences because both the spatial resolution and the temporal length contribute to the total number of tokens [22, 46]. As a result, full attention becomes one of the dominant bottlenecks in latency.

Existing acceleration methods for diffusion transformers mainly include quantization [12, 41, 42, 50], distillation [6, 7, 38, 39], caching-based methods [1, 5, 16, 18] and sparse attention [2–4, 14, 33, 36, 49]. Quantization reduces cost by lowering numerical precision and memory traffic. Distillation accelerates inference by transferring the behavior of the original model to a faster student, often with fewer denoising steps. Caching-based methods exploit temporal redundancy across denoising steps by either reusing cached features or predicting current features from previous cached states. Sparse attention, in contrast, targets the attention bottleneck itself by avoiding redundant attention interactions. In this work, we focus on sparse attention.

Sparse attention methods can be broadly divided into training-based [32, 40, 43, 45] and training-free approaches [2–4, 14, 33, 36, 49]. Training-based methods learn sparse patterns or sparse operators during training or fine-tuning, whereas training-free methods introduce sparsity only at inference time without modifying model weights. Our work focuses on the training-free setting, which is especially attractive in practice because it can be applied directly to pretrained Video DiTs. Sparse attention reduces computation by limiting how many keys and values each query actually attends to [35, 44]. In practical systems, sparse selection is usually performed at the block level rather than the individual-token level, since block-wise sparsity is much easier to execute efficiently on modern hardware [33, 36, 46].

Existing training-free sparse attention methods for Video DiTs can be roughly divided into offline and online approaches. Offline methods [4, 14, 51] determine sparse patterns in advance, but fixed patterns often generalize poorly across different prompts. Online methods [35, 36, 44] adapt the sparse pattern to the current input and are usually more flexible, but this comes with an extra cost: the sparse pattern itself must be predicted during inference. Within online methods, a key design choice is how many blocks each attention head should retain. Top-k methods [34, 49] retain a fixed number of blocks, but this ignores the fact that different heads can have very different intrinsic sparsity requirements. A fixed k may therefore waste computation on some heads while pruning too aggressively on others. For this reason, recent work increasingly adopts top-p sparsification [35, 36], which keeps blocks until a cumulative importance threshold is reached. However, even top-p methods usually apply a shared threshold across heads. This shared-threshold design can be suboptimal under a global sparsity budget, since heads can exhibit markedly different threshold-induced sparsity and error responses and the same cumulative threshold need not yield the best global allocation.

These observations point to two practical bottlenecks in existing training-free sparse attention for Video DiTs. The first is the online overhead of sparse-pattern prediction: if the mask is recomputed too often, the prediction overhead itself becomes expensive. The second is the suboptimality of shared head thresholds under a global sparsity budget: even when top-p is used, a single threshold does not account for heterogeneous threshold-induced sparsity and error responses across heads. Current studies only partially address these issues. For the first, existing methods often reduce mask-update cost through hand-crafted reuse schedules, such as refreshing the sparse pattern at fixed denoising intervals [34] or predicting masks only in part of the trajectory and reusing them afterwards [51]. While such strategies lower online overhead, they are prompt-agnostic and head-agnostic, and therefore cannot adapt to the actual mask evolution of different heads across denoising steps. For the second, most existing calibration methods are developed for top-k-style sparsification [19, 34, 48], where the goal is to calibrate sparsity levels or the number of retained blocks. In contrast, under top-p sparsification, the key control variable is the cumulative threshold itself, yet explicit calibration of head-wise top-p thresholds remains much less explored. As a result, current top-p methods typically still rely on shared thresholds, leaving room for improvement under a global sparsity budget.

We address these two issues with a head-wise sparse-attention framework consisting of two plug-in components. First, we propose **Temporal Mask Reuse (TMR)**, which reduces online mask-prediction overhead by deciding, for each head and denoising step, whether the cached sparse mask from the most recent refresh step can still be reused or a new mask should be predicted. The decision is made using a lightweight query-key drift statistic between the current step and the most recent mask-refresh step, so TMR adapts refresh frequency to the actual evolution of each head instead of following a fixed reuse schedule. Second, we propose **Error-guided Budgeted Calibration (EBC)**, which evaluates the model-output error induced by candidate thresholds and selects head-specific top-p thresholds under a target global sparsity budget. Specifically, for each head we measure multiple threshold candidates offline, and then solve a budgeted assignment problem to select one operating point per head. Our calibration objective is based on denoising-velocity error and further incorporates a weighted four-band 3D-FFT decomposition to better reflect the different impact of errors in different spatiotemporal frequency regions.

The two components are complementary. TMR is designed to reduce the online overhead of mask prediction, while EBC is designed to provide a more flexible allocation of the global sparsity budget across heterogeneous heads. Both can be inserted into existing top-p sparse-attention pipelines without changing pretrained weights or the underlying sparse kernel. Across Wan2.1-1.3B and Wan2.1-14B at 480P, our method consistently improves both XAttention and SVG2 in terms of both generation quality and inference speed. On Wan2.1-1.3B, it improves XAttention from 75.89% to 76.51% in VBench and increases the speedup from 1.30× to 1.49×. On Wan2.1-14B, it improves XAttention from 77.18% to 77.91% in VBench and increases the speedup from 1.22× to 1.30×. Similar gains are observed for SVG2: the VBench score increases from 76.79% to 77.00% on Wan2.1-1.3B and from 77.74% to 78.24% on Wan2.1-14B, while the speedup improves from 1.15× to 1.25× and from 1.11× to 1.17× on the two models, respectively. In our evaluated settings, the proposed method yields a better practical speed-quality trade-off than the compared top-p baselines.

Our main contributions are summarized as follows:

1. We provide a systematic analysis of training-free top-p sparse attention in Video DiTs and identify two under-explored forms of head-wise heterogeneity: temporal mask-evolution heterogeneity across denoising steps and threshold-response heterogeneity under top-p sparsification. These observations reveal why fixed mask-refresh schedules and shared top-p thresholds are suboptimal for practical speed–quality trade-offs.

2. Motivated by these observations, we propose a head-wise adaptive sparse-attention framework with two complementary components: Temporal Mask Reuse (TMR), which reduces online sparse-mask prediction overhead through adaptive mask reuse across denoising steps, and Error-guided Budgeted Calibration (EBC), which selects head-specific top-p thresholds according to measured model-output error rather than relying on a shared threshold or sparsity-only allocation. The framework is training-free and can be integrated into existing top-p sparse-attention pipelines without modifying pretrained weights or sparse kernels.

3. We instantiate the proposed framework on representative top-p sparse-attention baselines, including XAttention and SVG2, and conduct extensive experiments on Wan2.1-1.3B and Wan2.1-14B. Results show consistent improvements in the speed–quality trade-off, with more pronounced efficiency gains at higher resolution.

2 Related Work

2.1 Training-Based Sparse Attention for Video DiTs

A large line of work accelerates video diffusion transformers through trainable sparse attention, where the sparse pattern or attention operator is learned during training or fine-tuning rather than introduced only at inference time. Representative examples include video sparse attention and sparse-linear attention variants [32, 43, 45]. These methods often achieve strong efficiency-quality trade-offs because the model can adapt its parameters to the imposed sparse structure. However, they generally require additional optimization, retraining, or architectural modification, which limits their applicability when only pretrained checkpoints are available. Our work instead focuses on the training-free setting, where sparse attention can be introduced at inference time without modifying model weights.

2.2 Training-Free Sparse Attention for Video DiTs

Training-free sparse attention has emerged as a practical route for accelerating pretrained video diffusion transformers at inference time. Representative examples include Sparse VideoGen (SVG), which performs online profiling to identify sparse head patterns [33]; Sparse VideoGen2 (SVG2), which improves training-free sparse attention through semantic-aware token clustering [36]; and XAttention, which measures block importance via anti-diagonal scoring and further refines sparsity thresholds with dynamic programming [35]. Other training-free sparse attention methods for video generation include [2–4, 7, 13, 14, 20, 24, 27, 44, 49]. Our work operates in this training-free regime and focuses on two aspects beyond the sparse kernel itself: when a head should refresh its mask across denoising steps, and how a global sparsity budget should be distributed across heterogeneous heads.

Temporal Mask Reuse Across Denoising Steps. To reduce the additional online overhead introduced by sparse-pattern prediction, recent studies reuse sparse attention masks across some denoising steps rather than recomputing them at every step. LiteAttention shows that diffusion attention exhibits substantial temporal coherence and propagates skip decisions across denoising steps to reduce repeated profiling overhead [25]. AdaSpa exploits the observed cross-step invariance of sparse patterns and updates only at selected denoising steps, following a uniformly spaced refresh schedule [34]. PAROAttention predicts sparse masks normally in the earlier denoising steps and then reuses a common mask over the remaining steps to reduce overhead [51]. These methods collectively show that sparse masks need not be refreshed equally often across the denoising trajectory. Our Temporal Mask Reuse (TMR) is most closely related to this line, but differs in both control granularity and decision signal.

Offline Calibration. Another question is how sparsity should be allocated under a fixed global budget. Several recent works have explored this question through offline profiling. SVOO explicitly performs offline layer-wise sensitivity profiling to derive intrinsic per-layer pruning levels, and then applies online bidirectional co-clustering during inference [20]. PASA studies allocation from a different perspective: instead of calibrating sparsity across layers or heads, it profiles the denoising trajectory and uses a curvature-aware dynamic budgeting mechanism to allocate more exact computation to critical timesteps [48]. AdaSpa is also closely related because it emphasizes heterogeneity across heads and uses head-adaptive search to mitigate wasteful computation [34]. Among existing methods, XAttention is particularly relevant to our work because it explicitly refines sparse thresholds using a dynamic-programming-based procedure over candidate configurations [35]. However, this dynamic-programming-based refinement strategy is not suitable for Video DiTs with multi-step denoising. Our method is motivated by the same need for more precise sparsity allocation, but differs in both objective and formulation.

3 Key Observations

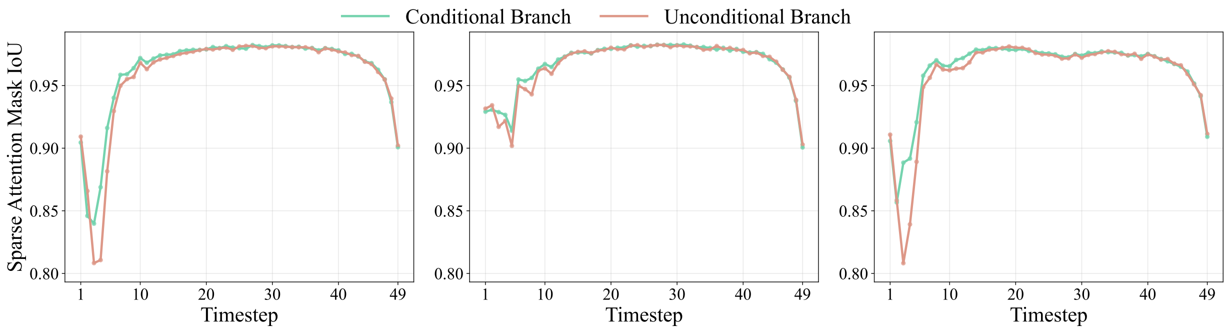
3.1 Observation I: Head-Wise Heterogeneity in Mask Evolution

In online sparse attention, sparse patterns are not predetermined but dynamically predicted during inference, introducing additional overhead [3, 34–36]. This raises a natural question: is it necessary to recompute the sparse mask at every denoising step, or can a recently predicted mask be safely reused? To answer this, we examine how sparse masks evolve across denoising steps and whether their temporal stability is consistent across prompts, layers, and heads.

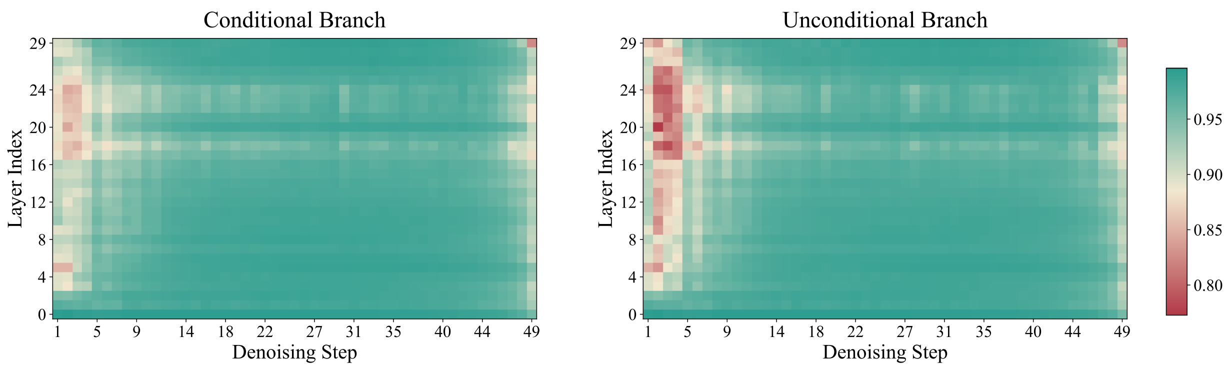
For this analysis, we compute the full attention score matrix and derive a token mask from the normalized attention probabilities. For head h, let $A_t^{(h)}(i,:) = \mathrm{softmax}(S_t^{(h)}(i,:))$ denote the attention distribution of query token i at denoising step t. We define the corresponding binary attention mask as $M_t^{(h)}(i,j) = \mathbf{1}\Big[j \in \mathcal{T}_{0.95}\big(A_t^{(h)}(i,:)\big)\Big]$ where $\mathcal{T}_{0.95}(\cdot)$ returns the minimal set of key indices whose cumulative probability mass reaches 0.95. Adjacent-step mask similarity is then measured by

$$\mathrm{IoU}_t^{(h)} = \frac{|M_t^{(h)} \cap M_{t+1}^{(h)}|}{|M_t^{(h)} \cup M_{t+1}^{(h)}|}. \tag{1}$$

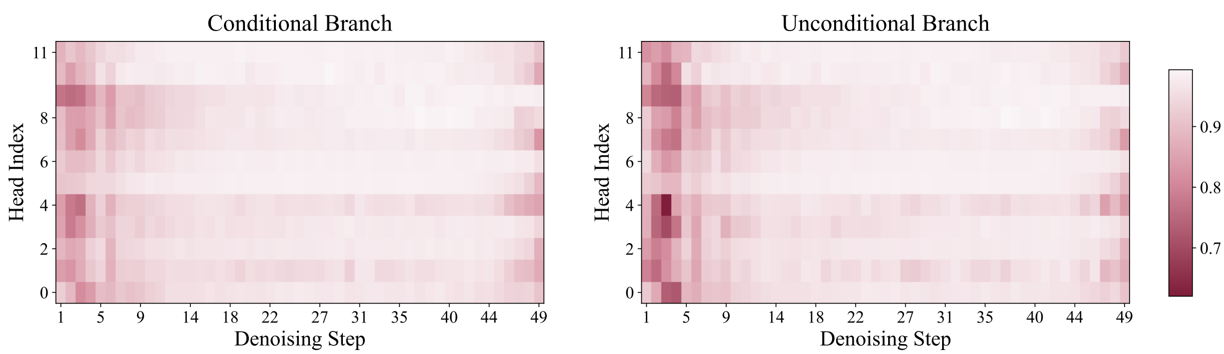
We visualize temporal mask similarity at three granularities. At the prompt level, IoU is averaged over all layers and heads for each prompt; at the layer level, it is averaged over heads within each layer; and at the head level, individual heads in a selected layer are shown without averaging. As shown in Figure 2, mask



(a) Prompt-level temporal mask similarity



(b) Layer-level temporal mask similarity



(c) Head-level temporal mask similarity

Figure 2. Temporal mask similarity is heterogeneous across prompts, layers, and heads. (a) Prompt-level curves for randomly sampled prompts still vary substantially after averaging over layers. (b) A layer-step mask-IoU heatmap for one randomly sampled prompt shows that the same heterogeneity also appears across layers. (c) Per-head mask-IoU heatmap within a randomly selected layer shows that this variation persists even inside one layer. Together, these observations motivate adaptive head-wise mask reuse instead of fixed shared reuse schedules.

stability is highly heterogeneous: prompt-level curves vary substantially even after aggregation, layer-level heatmaps reveal different stability patterns across layers within the same prompt, and head-level heatmaps show that such variation further persists across heads.

These observations indicate that predefined reuse schedules, such as fixed-interval refresh or reusing masks after a preset denoising step, are insufficiently adaptive. Since temporal stability varies not only across prompts and layers but also across heads within the same layer, mask reuse should be controlled at the head level according to the actual mask evolution, rather than imposed by a global schedule. Although our measurements are based on adjacent-step similarity, they provide direct motivation for reusing masks over multiple denoising steps when the corresponding head remains stable.

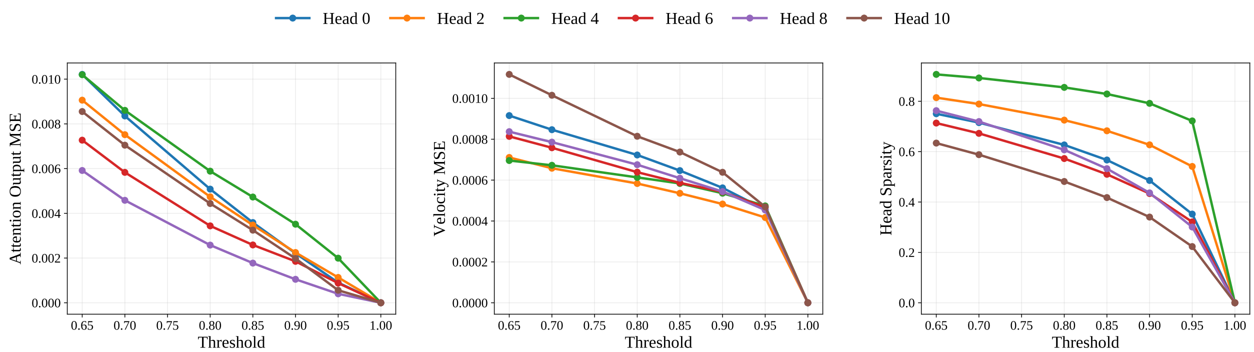


Figure 3. Head-wise threshold-induced sparsity and error response curves on Wan2.1-1.3B with XAttention [35], measured at a randomly selected layer and denoising timestep. Each polyline contains seven operating points corresponding to top-p thresholds $\{1.00, 0.95, 0.90, 0.85, 0.80, 0.70, 0.65\}$. From left to right, the panels show attention-output MSE, model-output MSE on denoising velocity, and head sparsity, all as functions of the threshold. Different heads follow clearly different threshold-induced sparsity and error response trajectories and therefore prefer different operating points.

3.2 Observation II: Head-Wise Heterogeneity in Top-p Threshold Responses

In top-p sparse attention, each head is controlled by a cumulative-mass threshold τ: candidate blocks are ranked by importance and retained until their accumulated importance reaches τ. Unlike top-k, where the number of retained blocks is fixed, the realized retention level of top-p depends on the score distribution itself. Flatter distributions require more blocks to reach the same cumulative mass, whereas more peaked distributions require fewer. Thus, although top-p is more adaptive than top-k, a shared threshold can still be suboptimal across heterogeneous heads.

For a head (l, h), a threshold τ does not directly specify a fixed sparsity level. Instead, it induces a head-dependent realized sparsity and approximation error. We therefore refer to each candidate threshold together with its measured outcomes as a *threshold-induced operating point*. Formally, for head (l, h) and threshold τ_k, the corresponding operating point is defined as

$$o_{l,h,k} = \big(\tau_k,\ s_{l,h}(\tau_k),\ e_{l,h}(\tau_k)\big),$$

where $s_{l,h}(\tau_k)$ denotes the realized sparsity and $e_{l,h}(\tau_k)$ denotes the induced approximation error. The error term can be instantiated with different measures, such as local attention-output error or final model-output error. Varying τ_k therefore produces a set of operating points, which together form a threshold-induced sparsity–error trade-off curve.

Figure 3 visualizes these curves on Wan2.1-1.3B with XAttention [35], using a randomly selected layer and denoising timestep. The resulting curves are clearly misaligned across heads. Even within the same layer and timestep, the same top-p threshold can produce substantially different sparsity levels and approximation errors. Some heads tolerate aggressive threshold reduction with little error increase, whereas others are much more sensitive. This heterogeneity appears in both attention-output MSE and model-output MSE, indicating that heads differ not only in their achievable sparsity but also in their threshold-induced error responses. These observations make a shared top-p threshold inherently restrictive.

4 Method

4.1 Overview

Figure 4 gives an overview of the proposed framework. The framework consists of two complementary head-level adaptation components designed to augment, rather than replace, existing top-p sparse-attention pipelines.

Temporal Mask Reuse (TMR) decides, for each head, whether to reuse the mask from the most recent refresh step or to predict a new one at the current step. It operates online through a lightweight query-key

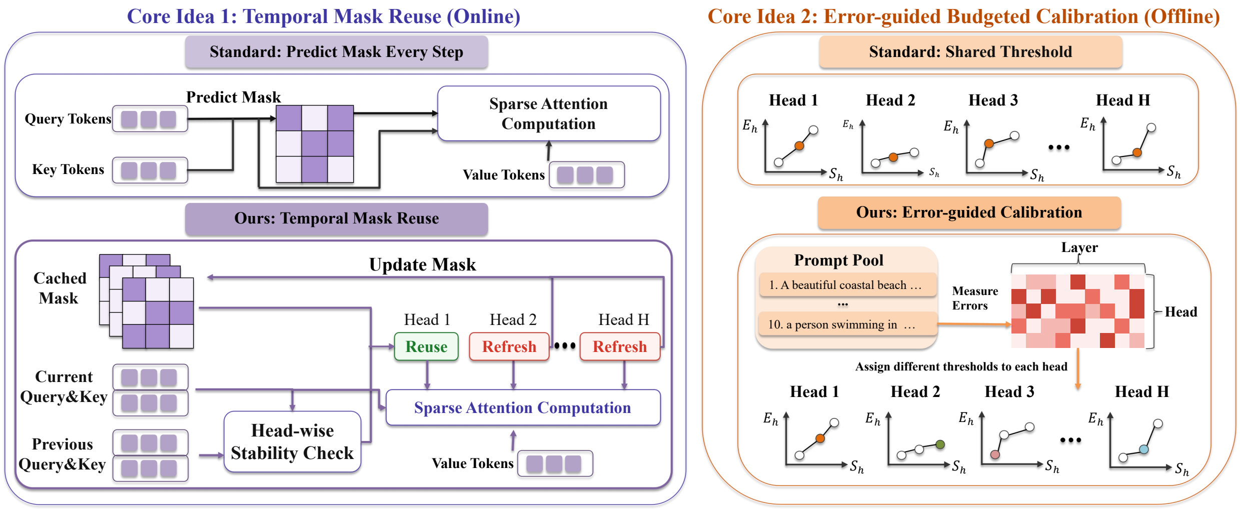


Figure 4. Overview of the proposed head-wise sparse-attention framework. Left: Temporal Mask Reuse (TMR) reduces online mask-prediction overhead by deciding, for each head, whether to reuse the previous sparse mask or refresh it using a lightweight query-key stability signal. Right: Error-guided Budgeted Calibration (EBC) operates offline, measuring candidate operating points for each head and selecting head-specific thresholds under a fixed global sparsity budget. The two components are complementary: TMR reduces online overhead, while EBC improves the global speed–quality trade-off without retraining or modifying the sparse-attention kernel.

temporal stability statistic and is applied during the mask computation stage of an existing top-p pipeline.

Error-guided Budgeted Calibration (EBC) determines the operating point of each attention head. It operates offline by measuring candidate operating points and selecting one calibrated operating point per head under a global sparsity budget, while leaving the underlying top-p sparse-attention mechanism unchanged.

The two components are designed to address different inefficiencies. TMR reduces repeated mask-prediction overhead during denoising, whereas EBC improves quality by allocating the same sparsity budget more effectively across heterogeneous heads.

4.2 Temporal Mask Reuse

Observation 3.1 shows that mask stability varies substantially across heads and denoising steps, making a shared reuse schedule overly restrictive. We therefore introduce *Temporal Mask Reuse* (TMR), a head-wise mechanism that decides whether the sparse mask of each head should be refreshed at the current denoising step or reused from a recent anchor step.

For each attention head h, TMR maintains an anchor step t_a, which denotes the most recent denoising step at which the sparse mask of this head is refreshed. At a later current step $t_b > t_a$, TMR decides whether the cached mask $M^{(h)}_{t_a}$ can still be reused or a new mask $M^{(h)}_{t_b}$ should be predicted. Let $Q^{(h)}_{t_a}, K^{(h)}_{t_a}, Q^{(h)}_{t_b}, K^{(h)}_{t_b} \in \mathbb{R}^{N\times D}$ denote the query and key matrices of head h at the anchor step t_a and the current step t_b, where N is the token sequence length and D is the head dimension. We use $q^{(h)}_{t,i}$ and $k^{(h)}_{t,i}$ to denote the i-th row of $Q^{(h)}_t$ and $K^{(h)}_t$, respectively. To decide whether the anchor mask can be reused at step t_b, TMR requires a lightweight proxy for cross-step mask stability. Directly comparing full attention scores or constructing the current sparse mask would be expensive. We therefore use query-key drift as the proxy. The rationale is that sparse masks are selected from block-importance scores induced by query-key interactions; if the query and key features of a head change only mildly between t_a and t_b, the induced block scores and the resulting sparse mask are also expected to remain stable. This leads to the following averaged token-wise query-key drift:

$$\tilde{d}^{(h)}_{t_a\to t_b} := \frac{1}{N}\sum_{i=1}^{N}\left\|q^{(h)}_{t_a,i} - q^{(h)}_{t_b,i}\right\|_1 + \frac{1}{N}\sum_{j=1}^{N}\left\|k^{(h)}_{t_a,j} - k^{(h)}_{t_b,j}\right\|_1 . \tag{2}$$

Practical sparse-attention systems usually make compute/skip decisions at the block level rather than the token-pair level, since block-wise sparsity better matches dense kernel execution and avoids highly irregular memory access [8, 33, 36, 46]. Let $\mathcal{B} = \{B_1, \ldots, B_M\}$ denote the set of candidate blocks for this head, where each block represents the interaction between a group of query tokens and a group of key/value tokens, and M is the total number of candidate blocks. Let $a_t(B)$ be the block importance score assigned by the underlying sparse-attention pipeline to block $B \in \mathcal{B}$ at step t. Let $S_t \subseteq \mathcal{B}$ be the retained block set selected by the top-p rule at step t, and define the changed-block ratio

$$R^{(h)}_{t_a \to t_b} := \frac{|S_{t_a} \triangle S_{t_b}|}{M}, \tag{3}$$

where $\triangle$ denotes the symmetric difference. This quantity measures the fraction of block-level compute/skip decisions that change between two denoising steps.

We formalize the connection between query-key drift and mask stability with the following proposition.

Proposition 1 (Query-key drift controls mask changes)**.** *We consider a single attention head h. Let t_a denote the most recent denoising step at which the sparse mask of this head is refreshed, and let $t_b > t_a$ denote the current denoising step. Then, the changed-block ratio between the anchor step t_a and the current step t_b is bounded by*

$$R^{(h)}_{t_a \to t_b} \leq C \tilde{d}^{(h)}_{t_a \to t_b}, \tag{4}$$

where $C > 0$ is a pipeline-dependent stability constant determined by the underlying sparse-attention method, such as its block scoring rule.

Proposition 1 provides a sufficient stability guarantee for TMR: smaller query-key drift implies a smaller upper bound on the changed-block ratio. This supports using query-key drift as a lightweight proxy for mask stability. When the query and key features of a head change only mildly between the anchor step and the current step, the induced block scores are expected to remain close to those at the anchor step, making the cached sparse mask more likely to remain valid for reuse. The detailed proof is provided in Appendix A.

In implementation, we use the cheaper mean-pooled statistic

$$d^{(h)}_{t_a \to t_b} = \left\| \bar{Q}^{(h)}_{t_a} - \bar{Q}^{(h)}_{t_b} \right\|_1 + \left\| \bar{K}^{(h)}_{t_a} - \bar{K}^{(h)}_{t_b} \right\|_1, \tag{5}$$

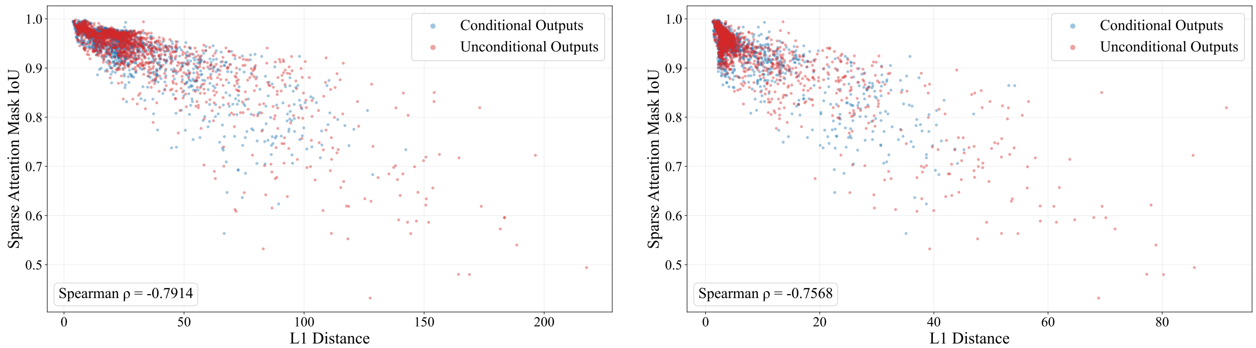
as a compressed proxy for $\tilde{d}^{(h)}_{t_a \to t_b}$. $\bar{Q}^{(h)}_t$ and $\bar{K}^{(h)}_t$ denote the token-averaged query and key features of head h at denoising step t, respectively. It reduces the cache cost from $O(LHND)$ to $O(LHD)$. For example, in Wan2.1-1.3B [28] 480P generation with classifier-free guidance, storing full-token query/key features in FP16 would require about 11.2 GB with $L = 30$, $H = 12$, $D = 128$, and $N = 32{,}760$, whereas storing the mean-pooled query/key features requires only about 0.35 MB. As shown in Figure 5, this lightweight proxy exhibits a similar empirical trend to the full-token drift while remaining much cheaper to cache.

If $d^{(h)}_{t_a \to t_b} \leq \delta$, where δ is a reuse threshold, mask prediction for head h is skipped and the previous sparse mask of that head is reused; otherwise a new mask is predicted for that head at the current step. The resulting procedure adds only lightweight per-head pooling and L_1 comparisons, while allowing mask-refresh decisions to vary across heads within different layers and denoising steps. Algorithm 1 summarizes the resulting online control flow.

Although TMR makes decisions at head granularity, sparse-mask construction may still involve non-negligible layer-level startup overhead. We therefore add a lightweight layer-level gating heuristic based on the fraction of heads marked for refresh in each layer. If this fraction is below a lower threshold, all heads in the layer reuse their cached masks; if it exceeds an upper threshold, the whole layer is refreshed; otherwise, the original head-wise TMR decisions are applied.

4.3 Error-guided Budgeted Calibration

Observation 3.2 shows that different heads exhibit distinct threshold-induced sparsity–error responses, making a shared top-p threshold restrictive. We therefore propose *Error-guided Budgeted Calibration*, which selects



(a) Full-token query-key drift **(b)** Mean-pooled query-key drift

Figure 5. Relationship between adjacent-step mask IoU and query-key drift. Both the full-token drift and the mean-pooled drift show clear negative correlations with mask IoU, indicating that the cheaper mean-pooled statistic preserves the predictive trend of the full-token drift while requiring much less cache memory. Each point corresponds to one sampled head-step pair from either conditional or unconditional branch.

Algorithm 1 Temporal Mask Reuse

Require: Current $Q_{t_b}^{(h)}, K_{t_b}^{(h)}$, reuse threshold δ, cached anchor features $\bar{Q}_{t_a}^{(h)}, \bar{K}_{t_a}^{(h)}$, cached anchor mask $M_{t_a}^{(h)}$

$\bar{Q}_{t_b}^{(h)} \leftarrow \text{mean}_N(Q_{t_b}^{(h)}), \quad \bar{K}_{t_b}^{(h)} \leftarrow \text{mean}_N(K_{t_b}^{(h)})$
if no cached anchor exists for head h **then**
  $M_{t_b}^{(h)} \leftarrow \textsc{PredictMask}(h, t_b)$
  Set anchor $t_a \leftarrow t_b$
  Cache $(\bar{Q}_{t_a}^{(h)}, \bar{K}_{t_a}^{(h)}, M_{t_a}^{(h)}) \leftarrow (\bar{Q}_{t_b}^{(h)}, \bar{K}_{t_b}^{(h)}, M_{t_b}^{(h)})$
else
  $d_{t_a \to t_b}^{(h)} \leftarrow \|\bar{Q}_{t_a}^{(h)} - \bar{Q}_{t_b}^{(h)}\|_1 + \|\bar{K}_{t_a}^{(h)} - \bar{K}_{t_b}^{(h)}\|_1$
  if $d_{t_a \to t_b}^{(h)} > \delta$ **then**
    $M_{t_b}^{(h)} \leftarrow \textsc{PredictMask}(h, t_b)$
    Set anchor $t_a \leftarrow t_b$
    Cache $(\bar{Q}_{t_a}^{(h)}, \bar{K}_{t_a}^{(h)}, M_{t_a}^{(h)}) \leftarrow (\bar{Q}_{t_b}^{(h)}, \bar{K}_{t_b}^{(h)}, M_{t_b}^{(h)})$
  else
    $M_{t_b}^{(h)} \leftarrow M_{t_a}^{(h)}$
    Keep the anchor cache unchanged
  end if
end if
return $M_{t_b}^{(h)}$

a head-specific top-p threshold under a target global sparsity budget. Unlike top-k calibration, where the calibrated variable directly determines the number of retained blocks, top-p calibration controls a cumulative-mass threshold whose realized sparsity depends on the head-specific score distribution. Thus, our goal is not simply to assign different sparsity levels, but to select, for each head, a threshold-induced operating point according to its measured sparsity and error.

Our calibration procedure is designed to satisfy three requirements. First, the offline measurement cost should remain manageable. Second, the calibration procedure should reduce overfitting to a particular prompt or timestep. Third, the calibration objective should provide a closer proxy to the impact of sparse attention. Accordingly, the calibration pipeline described below is constructed to satisfy these three requirements.

Consider a video DiT with L transformer layers and H attention heads per layer. For each head (l, h) and threshold τ, let $s_{l,h}(\tau)$ denote the realized sparsity and $e_{l,h}(\tau)$ denote the induced approximation error. Since the true joint model-output error under multi-head sparsification is generally not separable across heads, we adopt a first-order additive surrogate; Appendix B provides the derivation and discusses the approximation error. Under this surrogate, head-wise calibration can be formulated as

$$\min_{\{\tau_{l,h}\}} \sum_{l=1}^{L}\sum_{h=1}^{H} e_{l,h}(\tau_{l,h}) \quad \text{s.t.} \quad \frac{1}{LH}\sum_{l=1}^{L}\sum_{h=1}^{H} s_{l,h}(\tau_{l,h}) \geq S_{\min}, \tag{6}$$

where $S_{\min}$ is the target global sparsity. The solution assigns each head its own threshold instead of enforcing a single shared threshold.

In practice, $s_{l,h}(\tau)$ and $e_{l,h}(\tau)$ are not available in closed form. We therefore construct a discrete set of candidate operating points through offline measurement. For each head (l,h), we evaluate K candidate thresholds $\{\tau_1,\ldots,\tau_K\}$. Each threshold τ_k defines a threshold-induced operating point

$$o_{l,h,k} = \left(\tau_k, S_{l,h,k}, E_{l,h,k}\right),$$

where $S_{l,h,k}$ and $E_{l,h,k}$ are the measured sparsity and model-output error averaged over sampled prompt-step pairs. Let $x_{l,h,k} \in \{0,1\}$ indicate whether head (l,h) selects candidate k. The resulting integer linear program is

$$\begin{aligned} \min_{\{x_{l,h,k}\}} \quad & \sum_{l=1}^{L}\sum_{h=1}^{H}\sum_{k=1}^{K} E_{l,h,k}x_{l,h,k} \\ \text{s.t.} \quad & \sum_{k=1}^{K} x_{l,h,k} = 1, \quad \forall l,h, \\ & \frac{1}{LH}\sum_{l=1}^{L}\sum_{h=1}^{H}\sum_{k=1}^{K} S_{l,h,k}x_{l,h,k} \geq S_{\min}. \end{aligned} \tag{7}$$

This formulation selects exactly one operating point for each head while satisfying the global sparsity constraint.

To keep calibration lightweight, we first run the dense model on the prompt pool once and cache the dense denoising outputs. Measuring a candidate threshold for a given head then requires only one additional sparse forward using the cached dense reference, rather than replaying the full denoising trajectory. Moreover, each measurement sparsifies only one head while keeping all other heads dense, so measurements across heads are independent and can be parallelized.

To reduce overfitting to specific prompts or timesteps, we use both prompt sampling and interval-based timestep sampling. For each measurement, we sample prompts from a prompt pool instead of calibrating on a single input. Since the realized sparsity of the same head under the same threshold can still vary across denoising timesteps, as also observed in SpargeAttention [44] and in Figure 6, we divide the denoising trajectory into multiple intervals and sample one timestep from each interval. For a given head (l,h), all candidate thresholds are evaluated on the same sampled prompt-step pairs, and their measured errors and sparsities are averaged to obtain $E_{l,h,k}$ and $S_{l,h,k}$.

Following Observation 3.2, we measure calibration error at the model output rather than at the attention output. Attention-output MSE may not faithfully reflect the final effect of sparsifying a head, since perturbations can be attenuated or amplified by subsequent layers. In our setting, the model output is the predicted denoising velocity, so the calibration error is computed as the deviation between the sparse-output velocity and the dense-reference velocity.

Inspired by [17], we further examine whether velocity errors in different spatiotemporal frequency regions have the same effect on generated video quality. Starting from the dense denoising velocity y^{dense}, we perform a controlled frequency-domain perturbation study. For each frequency region Ω_q, we construct a perturbation whose Fourier coefficients are nonzero only inside Ω_q, transform it back to the original domain as Δ_q, and rescale it to the same relative magnitude:

$$\frac{\|\Delta_q\|_2}{\|y^{\text{dense}}\|_2} = \alpha, \tag{8}$$

where α is fixed for all frequency regions. We then add Δ_q to y^{dense} and evaluate the resulting video against the dense reference. This controlled setup ensures that differences in quality degradation reflect frequency sensitivity rather than perturbation magnitude.

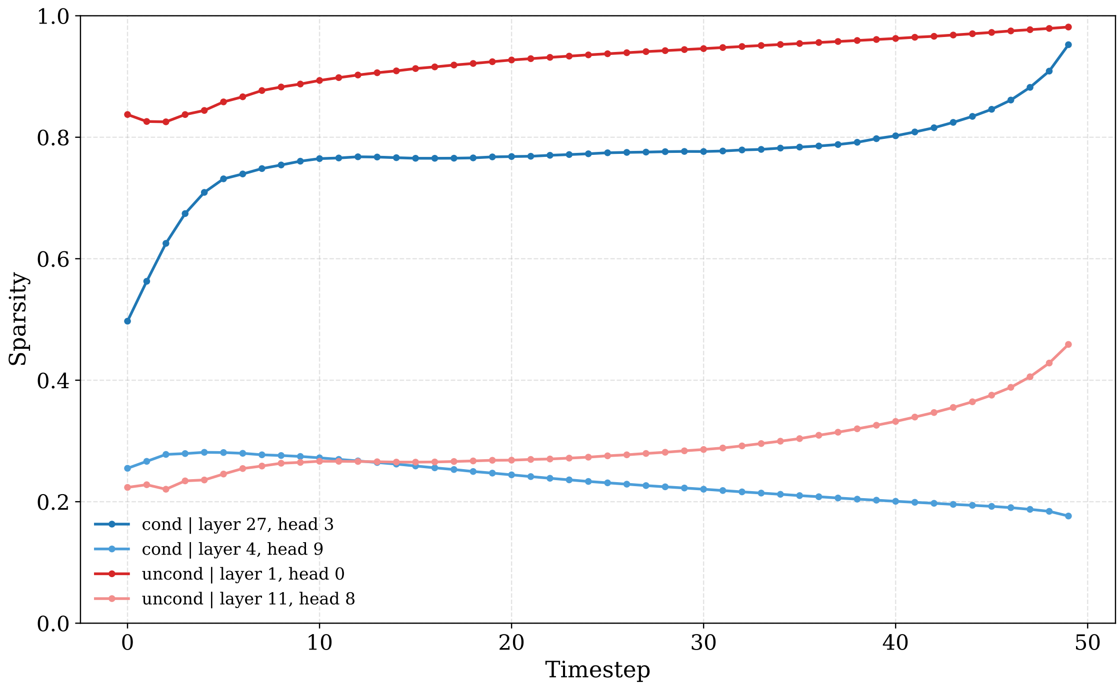


Figure 6. Realized sparsity over denoising timesteps for several randomly selected heads under a fixed sparsification threshold. Even for the same head, the achieved sparsity varies substantially across timesteps, while different heads exhibit different trajectories. This empirical observation supports interval-based timestep sampling in calibration, rather than measuring each head at only one denoising step.

As shown in Table 1, perturbations with the same relative magnitude can lead to different levels of degradation depending on their frequency region. In particular, perturbing the LL region is more harmful than perturbing the other regions in this study. This observation suggests that treating all velocity-space errors uniformly may be suboptimal for calibration. We therefore use a weighted four-band 3D-FFT objective as a frequency-aware calibration signal, while treating raw velocity-space MSE as an alternative objective in the ablation study.

Specifically, for head (l, h) under operating point τ, let

$$\epsilon_{l,h}(\tau) := y_{l,h}^{\text{sparse}}(\tau) - y^{\text{dense}}, \tag{9}$$

where $y_{l,h}^{\text{sparse}}(\tau)$ denotes the predicted denoising velocity when only head (l, h) is sparsified with operating point τ, and y^{dense} denotes the corresponding dense-reference velocity. We then compute a 3D FFT over the temporal and spatial dimensions of $\epsilon_{l,h}(\tau)$, and partition the shifted spectrum into four regions $\Omega_{\text{LL}}, \Omega_{\text{LH}}, \Omega_{\text{HL}}, \Omega_{\text{HH}}$ (where the first letter denotes the temporal-frequency band and the second letter denotes the spatial-frequency band; e.g., LH means low temporal frequency and high spatial frequency). For each frequency region, we define the band-wise normalized spectral error energy as

$$r_{l,h,q}(\tau) := \frac{\sum_{\omega \in \Omega_q} |\widehat{\epsilon}_{l,h}(\tau, \omega)|^2}{\sum_{\omega} |\widehat{y}^{\text{dense}}(\omega)|^2 + \varepsilon}, \qquad q \in \{\text{LL}, \text{LH}, \text{HL}, \text{HH}\}, \tag{10}$$

where ε is a small numerical constant used to stabilize the normalization. Here, $\widehat{\epsilon}_{l,h}(\tau, \omega)$ is the 3D-FFT of the model-output error $\epsilon_{l,h}(\tau)$ evaluated at frequency index ω, and $\widehat{y}^{\text{dense}}(\omega)$ is the 3D-FFT of the dense-reference denoising velocity. The numerator computes the spectral error energy restricted to region Ω_q, while the denominator normalizes it by the total spectral energy of the dense output. Thus, $r_{l,h,q}(\tau)$ measures the relative amount of model-output error falling into frequency band q.

Algorithm 2 Error-guided Budgeted Calibration

Require: Prompt pool $\mathcal{P}$, timestep intervals $\{\mathcal{I}_1, \dots, \mathcal{I}_J\}$, threshold candidates $\{\tau_1, \dots, \tau_K\}$, target sparsity $S_{\min}$

for all sampled prompts $p \in \mathcal{P}$ **do**
  run dense inference once and cache dense denoising outputs $\{y^{\text{dense}}(p,t)\}_t$
end for
for $l = 1$ to L **do**
  for $h = 1$ to H **do**
    assign one prompt $p_{l,h}$ to head (l,h)
    sample timesteps $\{t_j\}_{j=1}^{J}$ with $t_j \sim \mathcal{I}_j$
    for $k = 1$ to K **do**
      for $j = 1$ to J **do**
        sparsify only head (l,h) at $(p_{l,h}, t_j)$ with threshold τ_k
        compute model-output error $e_{l,h,k}^{(j)}$ from cached dense output and sparse output
        compute realized sparsity $s_{l,h,k}^{(j)}$
      end for
      $E_{l,h,k} \leftarrow \frac{1}{J}\sum_{j=1}^{J} e_{l,h,k}^{(j)}$
      $S_{l,h,k} \leftarrow \frac{1}{J}\sum_{j=1}^{J} s_{l,h,k}^{(j)}$
    end for
  end for
end for
solve the ILP defined by $\{E_{l,h,k}, S_{l,h,k}\}$ under sparsity budget $S_{\min}$
return calibrated operating points $\{\tau_{l,h}^{\star}\}$

Table 1. Sensitivity of generated-video quality to the same relative perturbations in different 3D-FFT regions of the dense denoising velocity. Under the same relative perturbation magnitude, disturbing the LL region causes the most severe degradation, whereas perturbations in other regions are less harmful, showing that equal error energy in different spatiotemporal frequency regions does not translate into equal video-quality impact. This provides empirical motivation for considering frequency-aware error weighting in calibration.

Region	PSNR↑	SSIM↑	LPIPS↓
LL (low temporal, low spatial)	28.19	0.8452	0.1024
LH (low temporal, high spatial)	29.58	0.8485	0.0970
HL (high temporal, low spatial)	29.85	0.8529	0.0818
HH (high temporal, high spatial)	30.35	0.8634	0.0791

The final frequency-aware calibration error is defined as

$$e_{l,h}(\tau) := \sum_{q \in \{\text{LL,LH,HL,HH}\}} w_q r_{l,h,q}(\tau), \tag{11}$$

where $w_q \geq 0$ controls the relative weight of each spatiotemporal frequency region. In our experiments, we assign larger weights to frequency regions that are empirically more sensitive to perturbations.

Together, model-output error measurement and 3D-FFT-based error decomposition are designed to better capture the output impact of sparse attention. Algorithm 2 summarizes the resulting offline calibration procedure.

4.4 Integration with Existing Top-p Pipelines

Both components act as plug-in modifications to existing top-p sparse-attention methods. They do not alter pretrained weights, the block-scoring rule, or the sparse kernel itself.

For TMR, any method that predicts or updates a sparse mask at every denoising step can insert the statistic check before mask construction. If the statistic indicates that a given head remains close to its previous step, the pipeline reuses that head's cached mask; otherwise it falls back to the original mask-prediction routine for that head. TMR therefore serves as a drop-in replacement for hand-crafted reuse schedules or always-refresh scheduling, while enabling finer-grained per-head refresh control.

For head-wise calibration, an offline stage measures candidate top-p settings for each head, solves the ILP once, and produces a calibrated table of per-head operating points. During online inference, the original method still performs top-p sparsification, but uses the calibrated head-specific setting instead of a single shared threshold. In this way, methods such as XAttention [35] and SVG2 [36] can be improved without changing their fundamental sparse-attention mechanism.

5 Experiments

5.1 Experimental Setup

Baselines. Experiments are conducted on pretrained video DiT backbones under fixed generation settings. Sparse attention is applied only at inference time, and all model weights remain frozen throughout calibration and inference. The method is instantiated on top of existing top-p baselines, in particular XAttention [35] and SVG2 [36]. For fair comparison, all methods are evaluated under the same attention-only optimization setting. We do not apply additional kernel optimizations to non-attention operators, and we do not apply dense warm-up steps that keep early denoising steps or early transformer layers in full attention.

Model setting. We evaluate on Wan2.1-1.3B and Wan2.1-14B [28]. Due to resource constraints, the main experiments are conducted at 480P, while we additionally report similarity metrics and speedup at 720P. The experiments follow a unified inference configuration. Specifically, each sample is generated with 81 frames and 50 denoising steps using the UniPC solver, with the shift parameter set to 8.0 and the guidance scale set to 6.0. All experiments are conducted on NVIDIA A800 PCIe GPU.

Evaluation metrics. Evaluation is conducted from three aspects: quality, similarity, and efficiency. We use VBench [10] for quality, PSNR [29]/SSIM [30]/LPIPS [47] for similarity, and latency/speedup for efficiency. Specifically, the reported VBench score is computed following the official VBench weighting protocol over six dimensions: subject consistency, motion smoothness, dynamic degree, aesthetic quality, imaging quality, and overall consistency.

Hyperparameters. The main experimental hyperparameters are summarized below. Unless explicitly stated otherwise, all experiments use these default settings. For Temporal Mask Reuse, the reuse threshold is set to 30.0 for XAttention [35] on both Wan2.1-1.3B and Wan2.1-14B, and to 8.0 for SVG2 [36]. For Error-guided Budgeted Calibration, the 50-step denoising trajectory is uniformly divided into four intervals, and one step is randomly sampled from each interval, so that each head is measured on four timesteps from its assigned prompt. Each sampled timestep is evaluated under three top-p thresholds, $0.85, 0.90, 0.95$. The resulting error and sparsity values are averaged over these four measurements to form $E_{l,h,k}$ and $S_{l,h,k}$, respectively. The model-output error is measured with the weighted 3D-FFT objective described in Section 4.3, using quadrant weights $(1.0, 0.5, 0.01, 0.01)$ for $(\mathrm{LL}, \mathrm{LH}, \mathrm{HL}, \mathrm{HH})$. To ensure a fair comparison with shared-threshold baselines, the global sparsity budget is chosen to be close to the realized sparsity obtained by XAttention [35] and SVG2 [36] under the common threshold 0.9.

5.2 Main Results

We compare the proposed method with the dense baseline and with representative top-p sparse attention methods, including XAttention [35] and SVG2 [36], under the same inference settings. Table 2 shows that the proposed head-wise enhancements improve both XAttention [35] and SVG2 [36] on Wan2.1-1.3B and Wan2.1-14B in our evaluation setting. Compared with their original versions, the enhanced variants achieve higher VBench scores and better similarity metrics, while also delivering higher speedup. On Wan2.1-1.3B, our method improves XAttention from 75.89% to 76.51% in VBench and increases the speedup from 1.30× to 1.49×; it also improves SVG2 from 76.79% to 77.00% in VBench, with the speedup increasing from 1.15× to 1.25×. The same trend is also visible on Wan2.1-14B, where both XAttention and SVG2 obtain better quality and similarity after applying our method, together with additional efficiency gains in terms of speedup.

These results indicate that the proposed components do not merely trade quality for speed. Instead, EBC improves the allocation of head-specific top-p thresholds under the same sparsity budget, while TMR translates the reduced mask-prediction overhead into additional end-to-end speedup.

Table 2. Main quantitative comparison among the original dense models, representative top-p sparse attention methods, and the corresponding top-p methods enhanced with our method.

Model	Method	Quality	Similarity			Efficiency	
		VBench↑	PSNR↑	SSIM↑	LPIPS↓	Latency↓	Speedup↑
Wan2.1-1.3B 480P	Dense	77.15%	–	–	–	195s	1.00×
	XAttention	75.89%	19.62	0.6219	0.2770	150s	1.30×
	XAttn + Ours	**76.51%**	**20.34**	**0.6431**	**0.2554**	**131s**	**1.49×**
	SVG2	76.79%	21.56	0.7189	0.1843	170s	1.15×
	SVG2 + Ours	**77.00%**	**21.61**	**0.7193**	**0.1785**	**156s**	**1.25×**
Wan2.1-14B 480P	Dense	79.31%	–	–	–	1030s	1.00×
	XAttention	77.18%	18.67	0.6801	0.2761	843s	1.22×
	XAttn + Ours	**77.91%**	**20.03**	**0.7084**	**0.2374**	**791s**	**1.30×**
	SVG2	77.74%	18.26	0.6487	0.3073	926s	1.11×
	SVG2 + Ours	**78.24%**	**19.14**	**0.6628**	**0.2635**	**882s**	**1.17×**

Table 3. Quantitative comparison at higher resolution on Wan2.1-1.3B at 720P.

Method	Similarity			Efficiency	
	PSNR↑	SSIM↑	LPIPS↓	Latency↓	Speedup↑
Dense	–	–	–	752s	1.00×
XAttention	**20.25**	0.6858	0.3260	441s	1.71×
XAttn + Ours	19.65	**0.7100**	**0.3212**	**389s**	**1.93×**
SVG2	24.86	0.8169	0.1663	475s	1.58×
SVG2 + Ours	**25.16**	**0.8210**	**0.1662**	**440s**	**1.71×**

5.3 Results at Higher Resolution

Due to resource constraints, the main quantitative comparison in this paper is conducted at 480P. However, at 480P the token count is still relatively limited, so attention does not yet dominate the end-to-end inference cost and the resulting speedup remains moderate. To further examine how the proposed method scales when the token count becomes larger, we additionally evaluate the same methods at 720P using Wan2.1-1.3B. Since this experiment is mainly intended to study the scaling of efficiency at higher resolution, Table 3 reports PSNR, SSIM, LPIPS, latency, and speedup, but does not include VBench.

As shown in Table 3, the efficiency benefit of the proposed method becomes more pronounced at 720P. On top of XAttention, our method reduces the latency from 441s to 389s and improves the speedup from 1.71× to 1.93×. In terms of similarity, it achieves higher SSIM and lower LPIPS, although PSNR decreases slightly. On top of SVG2, our method improves both similarity and efficiency, increasing the speedup from 1.58× to 1.71× while also slightly improving PSNR, SSIM, and LPIPS. Overall, these results suggest that as the resolution and token count increase, the proposed head-wise enhancements yield a more favorable efficiency gain and maintain a competitive quality–efficiency trade-off.

5.4 Ablation Studies

All ablation experiments are conducted on Wan2.1-1.3B at 480P using XAttention [35] under the same inference configuration as the main experiment.

5.4.1 Key Components Ablation

We isolate the contribution of the two proposed modules. Starting from the same underlying top-p sparse-attention baseline, we evaluate four variants: the baseline alone, the baseline with only Temporal Mask Reuse, the baseline with only Error-guided Budgeted Calibration, and the full method with both components enabled.

Table 4 isolates the effects of the two proposed components. Temporal Mask Reuse mainly contributes to efficiency improvement: compared with the baseline, enabling TMR alone increases the speedup from 1.30× to 1.49× and also improves the similarity metrics. However, its VBench score is slightly lower than that of

Table 4. Key-component ablation of the two proposed modules on Wan2.1-1.3B at 480P.

Variant	Quality	Similarity			Efficiency	
	VBench↑	PSNR↑	SSIM↑	LPIPS↓	Latency↓	Speedup↑
Baseline top-p	75.89%	19.62	0.6219	0.2770	150s	1.30×
+ TMR only	75.65%	20.45	0.6407	0.2533	131s	1.49×
+ EBC only	76.28%	20.38	0.6443	0.2496	150s	1.30×
Full method	76.51%	20.34	0.6431	0.2554	131s	1.49×

Table 5. Ablation of the TMR threshold δ on Wan2.1-1.3B at 480P.

δ	Setting	Mask Reuse Rate	Sparsity	Similarity			Efficiency	
				PSNR↑	SSIM↑	LPIPS↓	Latency↓	Speedup↑
0.0	No reuse	0.00%	74.25%	20.38	0.6443	0.2496	150s	1.30×
5.0	Conservative reuse	50.53%	74.09%	20.43	0.6436	0.2574	142s	1.37×
10.0	Moderate reuse	68.78%	73.83%	20.72	0.6529	0.2449	137s	1.42×
30.0	Strong reuse	87.48%	72.52%	20.34	0.6431	0.2554	131s	1.49×
100.0	Aggressive reuse	96.53%	61.62%	18.40	0.6325	0.2872	145s	1.34×

the baseline, suggesting that reusing masks mainly stabilizes the sparse attention pattern and makes the output closer to the reference trajectory, but does not by itself ensure better perceptual quality. In contrast, EBC mainly improves quality under the same computational budget: with EBC only, VBench increases from 75.89% to 76.28% while the speedup remains unchanged at 1.30×, and the similarity metrics also improve over the baseline. When the two components are combined, the full method achieves the strongest overall quality-efficiency trade-off among the variants in this table, reaching the highest VBench of 76.51% while preserving the larger 1.49× speedup brought by TMR. These results suggest that the two components play complementary roles: TMR is primarily responsible for reducing online overhead, whereas EBC is mainly responsible for improving quality.

5.4.2 Ablation on the Reuse Threshold in TMR

This section studies the effect of the reuse threshold δ in TMR. As shown in Table 5, the reuse threshold exhibits a non-monotonic effect. A moderate threshold, e.g., $\delta = 10.0$, improves all similarity metrics over no reuse. In contrast, conservative reuse slightly improves PSNR but slightly worsens SSIM and LPIPS, while an excessively large threshold leads to clear performance degradation.

This trend can be understood from two aspects. On the one hand, moderate reuse can suppress unnecessary mask updates caused by small fluctuations near the top-p selection boundary, thereby stabilizing the sparse pattern and reducing mask-prediction overhead. On the other hand, overly aggressive mask reuse does not simply skip mask prediction; it may also alter the realized sparsity along the denoising trajectory. Under a fixed top-p threshold, the realized sparsity of a head often changes as denoising proceeds. If a stale mask is reused for too long, the sparse pattern cannot adapt to this evolution, and the overall realized sparsity may drop substantially. As a result, aggressive reuse may reduce part of the mask-prediction cost, but its speedup gain can be limited because the attention computation itself is not reduced as much as expected. Meanwhile, by ignoring the actual evolution of the sparse mask, overly aggressive reuse also harms output quality.

5.4.3 Ablation on the Calibration Objective

This section studies the effect of the offline calibration objective. Specifically, we compare raw velocity-space MSE with the weighted four-band 3D-FFT objective used in the main method, while keeping the same sparsity budget and calibration pipeline. The goal is to examine whether incorporating spatiotemporal frequency sensitivity into the calibration objective can provide improvement in preserving the final model output. Table 6 shows that the weighted 3D-FFT objective produces the same PSNR, slightly higher SSIM, and comparable LPIPS to raw velocity MSE under the same sparsity budget. Overall, the difference between the two objectives remains small, but the SSIM gain suggests that the weighted spectral objective may provide a slightly better calibration signal for preserving structural consistency in the final output. One possible reason is that the

Table 6. Ablation of the calibration objective on Wan2.1-1.3B at 480P under the same sparsity budget.

Objective	Similarity		
	PSNR↑	SSIM↑	LPIPS↓
Velocity MSE	20.34	0.6388	0.2534
Weighted 3D-FFT	20.34	0.6431	0.2554

frequency-aware weighting places greater emphasis on errors in more important low-frequency regions, which may help retain coarse spatiotemporal structure and make the generated video globally closer to the dense reference. This interpretation remains consistent with the perturbation study in Table 1, which indicates that errors in different spatiotemporal frequency regions can have unequal effects on the final generated video.

6 Conclusion

In this paper, we presented two complementary head-level enhancements for training-free sparse attention in video diffusion transformers: Temporal Mask Reuse and Error-guided Budgeted Calibration. Temporal Mask Reuse introduces a lightweight query-key drift statistic for deciding, on a per-head basis, when mask prediction can be skipped and when a new mask should be generated. Error-guided Budgeted Calibration formulates per-head operating-point selection as a measurement-driven budgeted assignment problem, using a weighted four-band 3D-FFT deviation on denoising velocity as the primary model-output objective, and solves the resulting discrete optimization with an ILP. Taken together, the two components provide a practical path toward better speed-quality trade-offs for video generation.

References

[1] Jiazi Bu, Pengyang Ling, Yujie Zhou, Yibin Wang, Yuhang Zang, Dahua Lin, and Jiaqi Wang. Dicache: Let diffusion model determine its own cache. *arXiv preprint arXiv:2508.17356*, 2025.

[2] Aiyue Chen, Bin Dong, Jingru Li, Jing Lin, Kun Tian, Yiwu Yao, and Gongyi Wang. RainFusion: Adaptive Video Generation Acceleration via Multi-Dimensional Visual Redundancy, June 2025. URL http://arxiv.org/abs/2505.21036. arXiv:2505.21036 [cs].

[3] Aiyue Chen, Yaofu Liu, Junjian Huang, Guang Lian, Yiwu Yao, Wangli Lan, Jing Lin, Zhixin Ma, Tingting Zhou, and Harry Yang. RainFusion2.0: Temporal-Spatial Awareness and Hardware-Efficient Block-wise Sparse Attention, December 2025. URL http://arxiv.org/abs/2512.24086. arXiv:2512.24086 [cs].

[4] Pengtao Chen, Xianfang Zeng, Maosen Zhao, Peng Ye, Mingzhu Shen, Wei Cheng, Gang Yu, and Tao Chen. Sparse-vDiT: Unleashing the Power of Sparse Attention to Accelerate Video Diffusion Transformers, June 2025. URL http://arxiv.org/abs/2506.03065. arXiv:2506.03065 [cs].

[5] Liang Feng, Shikang Zheng, Jiacheng Liu, Yuqi Lin, Qinming Zhou, Peiliang Cai, Xinyu Wang, Junjie Chen, Chang Zou, Yue Ma, et al. Hicache: Training-free acceleration of diffusion models via hermite polynomial-based feature caching. *arXiv preprint arXiv:2508.16984*, 2025.

[6] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. *arXiv preprint arXiv:2410.12557*, 2024.

[7] Youping Gu, Xiaolong Li, Yuhao Hu, Minqi Chen, and Bohan Zhuang. BLADE: Block-Sparse Attention Meets Step Distillation for Efficient Video Generation, September 2025. URL http://arxiv.org/abs/2508.10774. arXiv:2508.10774 [cs].

[8] Junxian Guo, Haotian Tang, Shang Yang, Zhekai Zhang, Zhijian Liu, and Song Han. Block Sparse Attention. https://github.com/mit-han-lab/Block-Sparse-Attention, 2024.

[9] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[10] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, et al. Vbench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21807–21818, 2024.

[11] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, Junkun Yuan, Yanxin Long, Aladdin Wang, Andong Wang, Changlin Li, Duojun Huang, Fang Yang, Hao Tan, Hongmei Wang, Jacob Song, Jiawang Bai, Jianbing Wu, Jinbao Xue, Joey Wang, Kai Wang, Mengyang Liu, Pengyu Li, Shuai Li, Weiyan Wang, Wenqing Yu, Xinchi Deng, Yang Li, Yi Chen, Yutao Cui, Yuanbo Peng, Zhentao Yu, Zhiyu He, Zhiyong Xu, Zixiang Zhou, Zunnan Xu, Yangyu Tao, Qinglin Lu, Songtao Liu, Dax Zhou, Hongfa Wang, Yong Yang, Di Wang, Yuhong Liu, Jie Jiang, and Caesar Zhong. HunyuanVideo: A Systematic Framework For Large Video Generative Models, March 2025. URL http://arxiv.org/abs/2412.03603. arXiv:2412.03603 [cs].

[12] Muyang Li, Yujun Lin, Zhekai Zhang, Tianle Cai, Xiuyu Li, Junxian Guo, Enze Xie, Chenlin Meng, Jun-Yan Zhu, and Song Han. Svdquant: Absorbing outliers by low-rank components for 4-bit diffusion models. *arXiv preprint arXiv:2411.05007*, 2024.

[13] Qirui Li, Guangcong Zheng, Qi Zhao, Jie Li, Bin Dong, Yiwu Yao, and Xi Li. Compact Attention: Exploiting Structured Spatio-Temporal Sparsity for Fast Video Generation, August 2025. URL http://arxiv.org/abs/2508.12969. arXiv:2508.12969 [cs].

[14] Xingyang Li, Muyang Li, Tianle Cai, Haocheng Xi, Shuo Yang, Yujun Lin, Lvmin Zhang, Songlin Yang, Jinbo Hu, Kelly Peng, Maneesh Agrawala, Ion Stoica, Kurt Keutzer, and Song Han. Radial Attention: \$O(n\log n)\$ Sparse Attention with Energy Decay for Long Video Generation, December 2025. URL http://arxiv.org/abs/2506.19852. arXiv:2506.19852 [cs].

[15] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[16] Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It's time to cache for video diffusion model. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 7353–7363, 2025.

[17] Jiacheng Liu, Peiliang Cai, Qinming Zhou, Yuqi Lin, Deyang Kong, Benhao Huang, Yupei Pan, Haowen Xu, Chang Zou, Junshu Tang, et al. Freqca: Accelerating diffusion models via frequency-aware caching. *arXiv preprint arXiv:2510.08669*, 2025.

[18] Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Junjie Chen, and Linfeng Zhang. From reusing to forecasting: Accelerating diffusion models with taylorseers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15853–15863, 2025.

[19] Jiayi Luo, Jiayu Chen, Jiankun Wang, Cong Wang, Hanxin Zhu, Qingyun Sun, Chen Gao, Zhibo Chen, and Jianxin Li. Training-free sparse attention for fast video generation via offline layer-wise sparsity profiling and online bidirectional co-clustering. *arXiv preprint arXiv:2603.18636*, 2026.

[20] Jiayi Luo, Jiayu Chen, Jiankun Wang, Cong Wang, Hanxin Zhu, Qingyun Sun, Chen Gao, Zhibo Chen, and Jianxin Li. Training-Free Sparse Attention for Fast Video Generation via Offline Layer-Wise Sparsity Profiling and Online Bidirectional Co-Clustering, March 2026. URL http://arxiv.org/abs/2603.18636. arXiv:2603.18636 [cs].

[21] Guoqing Ma, Haoyang Huang, Kun Yan, Liangyu Chen, Nan Duan, Shengming Yin, Changyi Wan, Ranchen Ming, Xiaoniu Song, Xing Chen, Yu Zhou, Deshan Sun, Deyu Zhou, Jian Zhou, Kaijun Tan, Kang An, Mei Chen, Wei Ji, Qiling Wu, Wen Sun, Xin Han, Yanan Wei, Zheng Ge, Aojie Li, Bin Wang, Bizhu Huang, Bo Wang, Brian Li, Changxing Miao, Chen Xu, Chenfei Wu, Chenguang Yu, Dapeng Shi, Dingyuan Hu, Enle Liu, Gang Yu, Ge Yang, Guanzhe Huang, Gulin Yan, Haiyang Feng, Hao Nie, Haonan Jia, Hanpeng Hu, Hanqi Chen, Haolong Yan, Heng Wang, Hongcheng Guo, Huilin Xiong, Huixin Xiong, Jiahao Gong, Jianchang Wu, Jiaoren Wu, Jie Wu, Jie Yang, Jiashuai Liu, Jiashuo Li, Jingyang Zhang, Junjing Guo, Junzhe Lin, Kaixiang Li, Lei Liu, Lei Xia, Liang Zhao, Liguo Tan, Liwen Huang, Liying Shi, Ming Li, Mingliang Li, Muhua Cheng, Na Wang, Qiaohui Chen, Qinglin He, Qiuyan Liang, Quan Sun, Ran Sun, Rui Wang, Shaoliang Pang, Shiliang Yang, Sitong Liu, Siqi Liu, Shuli Gao, Tiancheng Cao, Tianyu Wang, Weipeng Ming, Wenqing He, Xu Zhao, Xuelin Zhang, Xianfang Zeng, Xiaojia Liu, Xuan Yang, Yaqi Dai, Yanbo Yu, Yang Li, Yineng Deng, Yingming Wang, Yilei Wang, Yuanwei Lu, Yu Chen, Yu Luo, Yuchu Luo, Yuhe Yin, Yuheng Feng, Yuxiang Yang, Zecheng Tang, Zekai Zhang, Zidong Yang, Binxing Jiao, Jiansheng Chen, Jing Li, Shuchang Zhou, Xiangyu Zhang, Xinhao Zhang, Yibo Zhu, Heung-Yeung Shum, and Daxin Jiang. Step-Video-T2V Technical Report: The Practice, Challenges, and Future of Video Foundation Model, February 2025. URL http://arxiv.org/abs/2502.10248. arXiv:2502.10248 [cs].

[22] Xin Ma, Yaohui Wang, Xinyuan Chen, Gengyun Jia, Ziwei Liu, Yuan-Fang Li, Cunjian Chen, and Yu Qiao. Latte: Latent diffusion transformer for video generation. *arXiv preprint arXiv:2401.03048*, 2024.

[23] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.

[24] Xuan Shen, Chenxia Han, Yufa Zhou, Yanyue Xie, Yifan Gong, Quanyi Wang, Yiwei Wang, Yanzhi Wang, Pu Zhao, and Jiuxiang Gu. DraftAttention: Fast Video Diffusion via Low-Resolution Attention Guidance, May 2025. URL http://arxiv.org/abs/2505.14708. arXiv:2505.14708 [cs].

[25] Dor Shmilovich, Tony Wu, Aviad Dahan, and Yuval Domb. LiteAttention: A Temporal Sparse Attention for Diffusion Transformers, November 2025. URL http://arxiv.org/abs/2511.11062. arXiv:2511.11062 [cs].

[26] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

[27] Wenhao Sun, Rong-Cheng Tu, Jingyi Liao, Zhao Jin, and Dacheng Tao. AsymRnR: Video Diffusion Transformers Acceleration with Asymmetric Reduction and Restoration, December 2024. URL http://arxiv.org/abs/2412.11706. arXiv:2412.11706 [cs].

[28] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wente Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han,

Zhi-Fan Wu, and Ziyu Liu. Wan: Open and Advanced Large-Scale Video Generative Models, April 2025. URL http://arxiv.org/abs/2503.20314. arXiv:2503.20314 [cs].

[29] Zhou Wang, Hamid R Sheikh, Alan C Bovik, et al. Objective video quality assessment. In *The handbook of video databases: design and applications*, volume 41, pages 1041–1078. CRC press Boca Raton, 2003.

[30] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004.

[31] Jianzong Wu, Liang Hou, Haotian Yang, Xin Tao, Ye Tian, Pengfei Wan, Di Zhang, and Yunhai Tong. Vmoba: Mixture-of-block attention for video diffusion models. *arXiv preprint arXiv:2506.23858*, 2025.

[32] Xinjian Wu, Hongmei Wang, Yuan Zhou, and Qinglin Lu. USV: Unified Sparsification for Accelerating Video Diffusion Models, December 2025. URL http://arxiv.org/abs/2512.05754. arXiv:2512.05754 [cs].

[33] Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, Jianfei Chen, Ion Stoica, Kurt Keutzer, and Song Han. Sparse VideoGen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity, April 2025. URL http://arxiv.org/abs/2502.01776. arXiv:2502.01776 [cs].

[34] Yifei Xia, Suhan Ling, Fangcheng Fu, Yujie Wang, Huixia Li, Xuefeng Xiao, and Bin Cui. Training-free and Adaptive Sparse Attention for Efficient Long Video Generation, February 2025. URL http://arxiv.org/abs/2502.21079. arXiv:2502.21079 [cs].

[35] Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. XAttention: Block Sparse Attention with Antidiagonal Scoring, March 2025. URL http://arxiv.org/abs/2503.16428. arXiv:2503.16428 [cs].

[36] Shuo Yang, Haocheng Xi, Yilong Zhao, Muyang Li, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Chenfeng Xu, Kelly Peng, Jianfei Chen, Song Han, Kurt Keutzer, and Ion Stoica. Sparse VideoGen2: Accelerate Video Generation with Sparse Attention via Semantic-Aware Permutation, October 2025. URL http://arxiv.org/abs/2505.18875. arXiv:2505.18875 [cs].

[37] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, Da Yin, Yuxuan Zhang, Weihan Wang, Yean Cheng, Bin Xu, Xiaotao Gu, Yuxiao Dong, and Jie Tang. CogVideoX: Text-to-Video Diffusion Models with An Expert Transformer, March 2025. URL http://arxiv.org/abs/2408.06072. arXiv:2408.06072 [cs].

[38] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Fredo Durand, and William T Freeman. Improved distribution matching distillation for fast image synthesis. *Advances in neural information processing systems*, 37:47455–47487, 2024.

[39] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6613–6623, 2024.

[40] Chenlu Zhan, Wen Li, Chuyu Shen, Jun Zhang, Suhui Wu, and Hao Zhang. Bidirectional Sparse Attention for Faster Video Diffusion Training, September 2025. URL http://arxiv.org/abs/2509.01085. arXiv:2509.01085 [cs].

[41] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread int4 quantization. *arXiv preprint arXiv:2411.10958*, 2024.

[42] Jintao Zhang, Jia Wei, Haofeng Huang, Pengle Zhang, Jun Zhu, and Jianfei Chen. Sageattention: Accurate 8-bit attention for plug-and-play inference acceleration. *arXiv preprint arXiv:2410.02367*, 2024.

[43] Jintao Zhang, Haoxu Wang, Kai Jiang, Shuo Yang, Kaiwen Zheng, Haocheng Xi, Ziteng Wang, Hongzhou Zhu, Min Zhao, Ion Stoica, Joseph E. Gonzalez, Jun Zhu, and Jianfei Chen. SLA: Beyond Sparsity in Diffusion Transformers via Fine-Tunable Sparse-Linear Attention, 2025. URL https://arxiv.org/abs/2509.24006. Version Number: 2.

[44] Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. SpargeAttention: Accurate and Training-free Sparse Attention Accelerating Any Model Inference, October 2025. URL http://arxiv.org/abs/2502.18137. arXiv:2502.18137 [cs].

[45] Peiyuan Zhang, Yongqi Chen, Haofeng Huang, Will Lin, Zhengzhong Liu, Ion Stoica, Eric Xing, and Hao Zhang. VSA: Faster Video Diffusion with Trainable Sparse Attention, October 2025. URL http://arxiv.org/abs/2505.13389. arXiv:2505.13389 [cs].

[46] Peiyuan Zhang, Yongqi Chen, Runlong Su, Hangliang Ding, Ion Stoica, Zhengzhong Liu, and Hao Zhang. Fast video generation with sliding tile attention. *arXiv preprint arXiv:2502.04507*, 2025.

[47] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018.

[48] Wentai Zhang, Ronghui Xi, Shiyao Peng, Jiayu Huang, Haoran Luo, Zichen Tang, et al. Ride the wave: Precision-allocated sparse attention for smooth video generation. *arXiv preprint arXiv:2604.12219*, 2026.

[49] Yuechen Zhang, Jinbo Xing, Bin Xia, Shaoteng Liu, Bohao Peng, Xin Tao, Pengfei Wan, Eric Lo, and Jiaya Jia. Training-Free Efficient Video Generation via Dynamic Token Carving, November 2025. URL http://arxiv.org/abs/2505.16864. arXiv:2505.16864 [cs].

[50] Tianchen Zhao, Tongcheng Fang, Haofeng Huang, Enshu Liu, Rui Wan, Widyadewi Soedarmadji, Shiyao Li, Zinan Lin, Guohao Dai, Shengen Yan, et al. Vidit-q: Efficient and accurate quantization of diffusion transformers for image and video generation. *arXiv preprint arXiv:2406.02540*, 2024.

[51] Tianchen Zhao, Ke Hong, Xinhao Yang, Xuefeng Xiao, Huixia Li, Feng Ling, Ruiqi Xie, Siqi Chen, Hongyu Zhu, Yichong Zhang, and Yu Wang. PAROAttention: Pattern-Aware ReOrdering for Efficient Sparse and Quantized Attention in Visual Generation Models, June 2025. URL http://arxiv.org/abs/2506.16054. arXiv:2506.16054 [cs].

Appendix

A Proof of Proposition 1

This appendix proves Proposition 1, which gives a sufficient condition under which a small anchor-current query-key drift leads to a small changed-block ratio. Specifically, for a head h, let t_a denote the most recent mask-refresh step and let $t_b > t_a$ denote the current step. The proof follows two steps. We first relate the query-key drift from t_a to t_b to block-score drift, and then relate block-score drift to the changed-block ratio.

Step 1: Query-key drift controls block-score drift. We follow the definition of the token-wise average query-key drift $\tilde{d}^{(h)}_{t_a\to t_b}$ in Eq. (2). We now show why this drift can control the average block-score drift used in Proposition 1.

Suppose that the block score at denoising step s is computed from block-level query and key summaries,

$$a_s(B) = \phi(u_s(B), v_s(B))\,, \tag{12}$$

where $s \in \{t_a, t_b\}$, $u_s(B), v_s(B) \in \mathbb{R}^D$ are the query and key summaries associated with block B, and ϕ is the block scoring rule used by the underlying sparse-attention pipeline.

Assume that ϕ is Lipschitz in its arguments over the regime of interest, since these scores are typically computed by continuous operations such as dot products, pooling, or other query-key interactions. That is, there exists a constant $C_\phi > 0$ such that

$$|\phi(u_1, v_1) - \phi(u_2, v_2)| \le C_\phi \left(\|u_1 - u_2\|_1 + \|v_1 - v_2\|_1\right). \tag{13}$$

For common block-level summarization operations, the block summaries inherit the stability of the underlying token features. Intuitively, a block summary is obtained by applying a bounded aggregation or reduction to the token features inside the corresponding query or key block; therefore, its variation from t_a to t_b cannot grow arbitrarily without a corresponding variation in the underlying tokens. We write this stability as

$$\frac{1}{M}\sum_{B\in\mathcal{B}} \|u_{t_a}(B) - u_{t_b}(B)\|_1 \le C_Q \frac{1}{N}\sum_{i=1}^{N} \|q^{(h)}_{t_a,i} - q^{(h)}_{t_b,i}\|_1, \tag{14}$$

and

$$\frac{1}{M}\sum_{B\in\mathcal{B}} \|v_{t_a}(B) - v_{t_b}(B)\|_1 \le C_K \frac{1}{N}\sum_{j=1}^{N} \|k^{(h)}_{t_a,j} - k^{(h)}_{t_b,j}\|_1. \tag{15}$$

Here, $C_Q, C_K > 0$ absorb the Lipschitz constants of the summarization rules and the block partition structure. Thus, these bounds are natural for common blockwise constructions based on averaging, pooling, or bounded local reductions.

Combining Eq. (12)–Eq. (15), we obtain

$$\begin{aligned}
\frac{1}{M}\sum_{B\in\mathcal{B}} |a_{t_a}(B) - a_{t_b}(B)| &\le \frac{C_\phi}{M}\sum_{B\in\mathcal{B}} \left(\|u_{t_a}(B) - u_{t_b}(B)\|_1 + \|v_{t_a}(B) - v_{t_b}(B)\|_1\right) && (16)\\
&\le C_\phi C_Q \frac{1}{N}\sum_{i=1}^{N} \|q^{(h)}_{t_a,i} - q^{(h)}_{t_b,i}\|_1 + C_\phi C_K \frac{1}{N}\sum_{j=1}^{N} \|k^{(h)}_{t_a,j} - k^{(h)}_{t_b,j}\|_1. && (17)
\end{aligned}$$

By absorbing constants into

$$L := C_\phi \max\{C_Q, C_K\},$$

we have

$$D^{(h)}_{t_a\to t_b} := \frac{1}{M}\sum_{B\in\mathcal{B}} |a_{t_a}(B) - a_{t_b}(B)| \le L\tilde{d}^{(h)}_{t_a\to t_b}. \tag{18}$$

Step 2: Block-score drift controls changed-block ratio. We next relate block-score drift to changes in the sparse mask. Let $S_{t_a} \subseteq \mathcal{B}$ and $S_{t_b} \subseteq \mathcal{B}$ denote the retained block sets selected by the top-p rule at the anchor step t_a and the current step t_b, respectively. Recall that the changed-block ratio is defined as

$$R^{(h)}_{t_a \to t_b} := \frac{|S_{t_a} \triangle S_{t_b}|}{M}. \tag{19}$$

Since the top-p sparse mask is determined by the block-score distribution, its stability is naturally tied to the stability of the underlying block scores. In particular, if the block scores change only mildly from t_a to t_b, the retained block set is expected to change only mildly as well. We formalize this intuition with a score-to-mask stability condition: there exists a constant $C_s > 0$, determined by the underlying sparse-attention pipeline, such that

$$R^{(h)}_{t_a \to t_b} \le C_s D^{(h)}_{t_a \to t_b}. \tag{20}$$

Here, C_s captures the stability of the score-to-mask mapping, including the block scoring rule and the top-p mask selection procedure. This condition states that smaller average block-score drift leads to a smaller upper bound on the fraction of changed block-level compute/skip decisions, without requiring an explicit characterization of the detailed top-p sorting behavior.

Finally, substituting Eq. (18) into Eq. (20) gives

$$R^{(h)}_{t_a \to t_b} \le C_s L \tilde{d}^{(h)}_{t_a \to t_b}. \tag{21}$$

Defining $C := C_s L$, we obtain

$$R^{(h)}_{t_a \to t_b} \le C \tilde{d}^{(h)}_{t_a \to t_b}. \tag{22}$$

This completes the proof.

The result provides a sufficient stability argument for Temporal Mask Reuse. A smaller anchor-current query-key drift leads to a smaller average block-score drift, which in turn limits the fraction of changed block-level compute/skip decisions under the score-to-mask stability condition. This supports using query-key drift as a lightweight proxy for mask stability. The empirical negative correlation between query-key drift and adjacent-step mask IoU in Figure 5 further validates this practical reuse signal.

B Additive Calibration Objective as a Local Surrogate

This appendix clarifies why the offline calibration objective is modeled as a head-wise additive objective. The key point is as follows. Exact additivity holds at the multi-head-attention output, because the perturbations introduced by different heads are linearly combined by the output projection. However, after these perturbations propagate through the remaining network and are mapped to the final scalar calibration error, the resulting joint error is generally *not* exactly additive. Therefore, the additive objective used in our calibration should be interpreted as a *local surrogate* that neglects cross-head interaction terms and higher-order effects, rather than as an exact factorization of the true full-network error.

Setup. Consider layer l with H attention heads. Let $o_{l,h}$ denote the output of head h before the output projection, and let

$$A_l = \mathrm{Concat}(o_{l,1}, \dots, o_{l,H}) W_O \tag{23}$$

be the multi-head attention output after the projection matrix W_O. Suppose sparse attention induces a perturbation $\delta_{l,h}$ on head h. Then the perturbed layer output is

$$A^{\mathrm{sparse}}_l = \mathrm{Concat}(o_{l,1} + \delta_{l,1}, \dots, o_{l,H} + \delta_{l,H}) W_O. \tag{24}$$

Therefore, the attention-output perturbation satisfies

$$\Delta A_l := A^{\mathrm{sparse}}_l - A_l = \mathrm{Concat}(\delta_{l,1}, \dots, \delta_{l,H}) W_O = \sum_{h=1}^{H} \delta_{l,h} W_{O,h}, \tag{25}$$

where $W_{O,h}$ denotes the block of W_O corresponding to head h. Define

$$\Delta A_{l,h} := \delta_{l,h} W_{O,h}, \tag{26}$$

so that

$$\Delta A_l = \sum_{h=1}^{H} \Delta A_{l,h}. \tag{27}$$

Exact additivity at the perturbation injection point. Equation (25) shows that, at the output of the multi-head attention module, head-wise sparse perturbations are exactly additive.

From layer perturbation to model-output perturbation. Let $\mathcal{F}_l$ denote the mapping from the attention output A_l of layer l to the final denoising output y. Around the dense operating point, assume that $\mathcal{F}_l$ is twice continuously differentiable. Then, for sufficiently small ΔA_l, a first-order Taylor expansion gives

$$y^{\text{sparse}} - y^{\text{dense}} = \mathcal{F}_l(A_l + \Delta A_l) - \mathcal{F}_l(A_l) = J_l \Delta A_l + R_l, \tag{28}$$

where J_l is the Jacobian of $\mathcal{F}_l$ at A_l, and R_l is the higher-order remainder term. If the Hessian of $\mathcal{F}_l$ is bounded in a neighborhood of A_l, then there exists a constant β_l such that

$$\|R_l\| \leq \frac{\beta_l}{2} \|\Delta A_l\|^2. \tag{29}$$

Substituting Eq. (25) into Eq. (28) yields

$$y^{\text{sparse}} - y^{\text{dense}} = \sum_{h=1}^{H} J_l \Delta A_{l,h} + R_l. \tag{30}$$

Thus, the model-output perturbation is additive to first order at the vector level, up to the higher-order remainder R_l.

From model-output perturbation to scalar calibration error. The calibration objective is a scalar error measured on the final model output, for example velocity-space MSE or the weighted spectral error.

For a fixed layer l, let $\{\Delta A_{l,h}\}_{h=1}^{H}$ denote the perturbations induced by sparsifying the attention heads in that layer, measured at the attention-output. Let $\mathcal{F}_l$ denote the mapping from the attention output of layer l to the final denoising output. Then

$$y^{\text{sparse}} = \mathcal{F}_l\left(A_l + \sum_{h=1}^{H} \Delta A_{l,h}\right), \qquad y^{\text{dense}} = \mathcal{F}_l(A_l).$$

We define the scalar calibration error associated with layer l as

$$e_l(\{\Delta A_{l,h}\}) := \phi\left(y^{\text{sparse}}, y^{\text{dense}}\right), \tag{31}$$

where $\phi(\cdot, \cdot)$ is an error functional that maps the difference between the sparse and dense outputs to a scalar quantity, such as velocity-space MSE or the weighted spectral error.

Equivalently, defining $\Delta y_l := y^{\text{sparse}} - y^{\text{dense}}$, we may write

$$e_l(\{\Delta A_{l,h}\}) = \psi(\Delta y_l), \tag{32}$$

for some nonnegative scalar function ψ with $\psi(0) = 0$.

For the squared-error-type objectives considered in this work, the leading variation around the dense point is naturally second order rather than first order. Assume that ψ is twice continuously differentiable around $\Delta y_l = 0$. Then, for some intermediate point ξ,

$$e_l(\{\Delta A_{l,h}\}) = \frac{1}{2} \Delta y_l^{\top} H_{\psi}(\xi) \Delta y_l, \tag{33}$$

where $H_\psi(\xi)$ is the Hessian of ψ evaluated at ξ.

Now define the first-order propagated perturbation of head h as

$$u_{l,h} := J_l \Delta A_{l,h}. \tag{34}$$

Using Eq. (30), we have

$$\Delta y_l = \sum_{h=1}^{H} u_{l,h} + R_l. \tag{35}$$

Ignoring the higher-order remainder R_l for the moment and substituting $\Delta y_l \approx \sum_h u_{l,h}$ into Eq. (33), we obtain the local quadratic approximation

$$e_l \approx \frac{1}{2}\left(\sum_{h=1}^{H} u_{l,h}\right)^\top H_\psi(\xi)\left(\sum_{h=1}^{H} u_{l,h}\right). \tag{36}$$

Expanding this expression gives

$$e_l \approx \frac{1}{2}\sum_{h=1}^{H} u_{l,h}^\top H_\psi(\xi) u_{l,h} + \frac{1}{2}\sum_{\substack{h,h'=1\\ h\neq h'}}^{H} u_{l,h}^\top H_\psi(\xi) u_{l,h'}. \tag{37}$$

Equation (37) shows that the true local scalar error contains two parts: (i) head-wise self terms, and (ii) cross-head interaction terms.

Additive surrogate and its interpretation. Equation (37) implies that the true joint scalar error is not exactly additive in general. A natural additive surrogate is obtained by dropping the interaction terms and retaining only the head-wise self contributions:

$$e_l^{\text{diag}} := \sum_{h=1}^{H} \widetilde{e}_{l,h}, \qquad \widetilde{e}_{l,h} := \frac{1}{2} u_{l,h}^\top H_\psi(\xi_h) u_{l,h}, \tag{38}$$

where ξ_h denotes a local point associated with head h. This surrogate should be viewed as a *diagonalized local approximation*: it keeps the per-head terms but neglects pairwise interactions and higher-order coupling effects.

In practice, our method does not explicitly compute $\widetilde{e}_{l,h}$ from local curvature. Instead, for each candidate operating point, we *measure* the isolated single-head error by sparsifying only one head while keeping the others dense. Denote this measured quantity by

$$E_{l,h,k} := \phi\Big(y_{l,h,k}^{\text{sparse}},\, y^{\text{dense}}\Big), \tag{39}$$

where k indexes the candidate operating point. The offline ILP then optimizes the measurement-driven additive surrogate

$$\sum_{l=1}^{L}\sum_{h=1}^{H}\sum_{k=1}^{K} E_{l,h,k} x_{l,h,k}, \tag{40}$$

which approximates the true joint objective by summing isolated single-head errors.

What is neglected by the surrogate. The gap between the true full-network joint error and the additive surrogate comes from three sources:

1. cross-head interaction terms within the same layer, as shown in Eq. (37);
2. higher-order propagation terms represented by R_l in Eq. (28);
3. cross-layer interactions, since the ILP sums measurements over all layers and heads while treating them as independently selectable operating points.

Therefore, the additive objective is not an exact decomposition of the full network error.

Practical implication for calibration. Our ILP should thus be interpreted as optimizing a measurement-driven local additive surrogate of the true joint objective. The additive calibration objective can be viewed as a principled and tractable surrogate for offline selection, rather than as a strict factorization of the full-network calibration error.